

ASM

Esercitazione
Architettura degli elaboratori

Esercizio 1

- Considerando che i valori 10, 5, 20, 100 siano memorizzati rispettivamente nei registri \$s0, \$s1, \$s2, \$s3, qual è il valore contenuto nel registro destinazione con l'esecuzione delle seguenti istruzioni:
 - add \$t0, \$s0, \$s1
 - add \$t1, \$s2, \$s3
 - sub \$t2, \$t1, \$t0
- Qual è la rappresentazione binaria delle istruzioni?

Esercizio 1 – soluzione (1/2)

- Considerando che

- \$s0 contiene 10
- \$s1 contiene 5
- \$s2 contiene 20
- \$s3 contiene 100
-

- Il valore contenuto nei registri destinazione delle istruzioni è

- add \$t0, \$s0, \$s1 **\$t0 = 15**
- add \$t1, \$s2, \$s3 **\$t1 = 120**
- sub \$t2, \$t1, \$t0 **\$t2 = 105**

Addition (with overflow)

add rd, rs, rt	0	rs	rt	rd	0	0x20
	6	5	5	5	5	6

Put the sum of registers rs and rt into register rd.

Subtract (with overflow)

sub rd, rs, rt	0	rs	rt	rd	0	0x22
	6	5	5	5	5	6

Put the difference of registers rs and rt into register rd.

Esercizio 1 – soluzione (2/2)

Per la rappresentazione binaria delle istruzioni servirà **ricavare i numeri dei registri dal nome simbolico** facendo riferimento alla tabella a pag A-24, App. A

Dalla tabella a pag A-24, App. A:

- \$t0 è il registro \$8 (01000 in binario)
- \$t1 è il registro \$9 (01001 in binario)
- \$t2 è il registro \$10 (01010 in binario)
- \$s0 è il registro 16 (10000 in binario)
- \$s1 è il registro 17 (10001 in binario)
- \$s2 è il registro 18 (10010 in binario)
- \$s3 è il registro 19 (10011 in binario)

add \$t0, \$s0, \$s1 [opcode=0 e func=32₁₀]

000000 10000 10001 01000 00000 100000

add \$t1, \$s2, \$s3 [opcode=0 e func=32₁₀]

000000 10010 10011 01001 00000 100000

sub \$t2, \$t1, \$t0 [opcode=0 e func=34₁₀]

000000 01001 01000 01010 00000 100010

Addition (with overflow)

add rd, rs, rt	0	rs	rt	rd	0	0x20
	6	5	5	5	5	6

Subtract (with overflow)

sub rd, rs, rt	0	rs	rt	rd	0	0x22
	6	5	5	5	5	6

Esercizio 2

Quale valore assumono i registri \$t0, \$t1, \$t2 dopo l'esecuzione del seguente frammento di codice?

```
.data 0x10010000
```

```
numeri: .word 1, 2, 3
```

```
.text
```

```
    lw $t0, numeri
```

```
    la $t1, numeri
```

```
    lw $t2, 8($t1)
```

Esercizio 2 – soluzione

Con l'esecuzione del frammento di codice

```
.data      0x10010000
```

```
numeri:    .word 1, 2, 3
```

```
.text
```

```
    lw $t0, numeri
```

#istruzione load word

```
    la $t1, numeri
```

#istruzione load address

```
    lw $t2, 8($t1)
```

avremo:

- in \$t0, la word presente all'indirizzo indicato dall'etichetta numeri e quindi **\$t0 = 1**
- in \$t1, l'indirizzo dell'etichetta numeri e quindi **\$t1=0x1001000** poichè numeri è il primo dei dati presenti nel data segment che inizia all'indirizzo specificato dalla direttiva .data
- in \$t2, la word che si trova 8 byte (2 word) dopo l'indirizzo uguale al contenuto di \$t1 e quindi **\$t2 = 3**

Load word

```
lw rt, address
```

0x23	rs	rt	Offset
6	5	5	16

Load the 32-bit quantity (word) at *address* into register *rt*.

Load address

```
la rdest, address
```

pseudoinstruction

Load computed *address*—not the contents of the location—into register *rdest*.

Esercizio 3

Raddoppiare i primi 37 valori (della dimensione di una word) memorizzati a partire dalla locazione di memoria il cui indirizzo è specificato dal contenuto del registro \$t0

Utilizziamo un ciclo per non riscrivere la stessa cosa 37 volte ...

Esercizio 3 – soluzione 1 (1/2)

addi \$t1, \$zero, 37	#\$t1 variabile di controllo del ciclo inizializzata a 37
perdue: lw \$t2, 0(\$t0)	#metto in \$t2 il primo valore, in memoria all'indirizzo contenuto in \$t0
add \$t2, \$t2, \$t2	#moltiplico per 2 sommando il contenuto del registro con se stesso
sw \$t2, 0(\$t0)	#salvo in memoria all'indirizzo da cui ho letto
addi \$t1, \$t1, -1	#decremento variabile controllo del ciclo (\$t1) di 1
beq \$t1, \$zero, fine	#se \$t1 ha raggiunto il valore 0, ho finito
addi \$t0, \$t0, 4	#incremento \$t0 di 4 byte → \$t0 "punta" a successiva word in memoria
j perdue	#altrimenti, esegue stesse istruzioni (su word successiva)
fine: ...	

N.B. Le istruzioni lw e sw sono di tipo I-type, l'offset è un valore costante e NON può essere sostituito con un registro a cui cambiamo il valore del contenuto

Esercizio 3 – soluzione 2 (2/2)

li \$t3, 37

#inizializzo il contatore delle word da leggere

ciclo: lw \$t2, 0(\$t0)
add \$t2, \$t2, \$t2
sw \$t2, 0(\$t0)
addi \$t0, \$t0, 4
addi \$t3, -1
bgtz \$t3, ciclo

#carico il valore in \$t2
#raddoppio il valore caricato
#salvo il valore raddoppiato
#incremento \$t0 per caricare la word successiva
#decremento il contatore delle word
#ripeto il ciclo se \$t3 è maggiore a zero

fine:

Branch on greater than zero

bgtz rs, label

7	rs	0	Offset
6	5	5	16

Conditionally branch the number of instructions specified by the offset if register *rs* is greater than 0.

Esercizio 4 (ciclo for)

Dato il frammento di codice assembly

.data

numeri: .word 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

risultato: .word 0

scrivere un programma assembly che calcoli la somma dei 10 numeri disponibili in memoria all'etichetta "numeri"

Esercizio 4 (ciclo for) – soluzione

Calcolare la somma di 10 numeri disponibili in memoria

```
.data
numeri: .word 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
risultato: .word 0
dim: .word 10

.text
lw $t6, dim          #numero di numeri
                      #da sommare

add $s0, $zero, $zero #somma = 0
add $t5, $zero, $zero #indice = 0
la $t7, numeri

ciclo: lw $t0, 0($t7)  #metto in $t0 il numero da
                      #sommare

add $s0, $s0, $t0 #aggiungo il numero in $t0 a $s0
addi $t7, $t7, 4   #mi sposto di 1 word (4 byte)
addi $t5, $t5, 1    #incremento indice (contatore
                      #del ciclo)

blt $t5, $t6, ciclo #se ho eseguito il ciclo meno
                    #di 10 volte, ripeto ciclo

sw $s0, risultato #terminato il ciclo, salvo $s0
                  #all'indirizzo specificato
                  #dall'etichetta risultato

Branch on less than

blt rsrc1, rsrc2, label
```

Conditionally branch to the instruction at the label if register `rsrc1` is less than `rsrc2`.

Esercizio 5 (if-then-else)

- Scrivere un programma assembly che legge 3 valori della dimensione di una word che si trovano in memoria all'etichetta "valori" e memorizza in memoria, nella locazione subito dopo il terzo valore:
 - il risultato della somma dei tre valori, se il primo valore è positivo
 - il prodotto dei tre numeri, se il primo valore è negativo
 - l'AND del secondo e del terzo numero, se il primo è uguale a zero

Esercizio 5 – soluzione

.data		
valori:	.word 1, 5, 2	#valori in memoria
	.word 0	#spazio in cui sara' salvato il risultato
.text		
main:	la \$t7, valori	#carico nel registro \$t7 l'indirizzo a cui si trova l'etichetta "valori" e quindi
		#del primo dato di dimensione di una word
	lw \$t0, 0(\$t7)	#carico in \$t0 il primo valore
	lw \$t1, 4(\$t7)	#carico in \$t1 il secondo valore (offset della load_word è 4 byte=1 word)
	lw \$t2, 8(\$t7)	#carico in \$t2 il terzo valore (offset della load_word è 8 byte=2 word)
	bgt \$t0, \$zero, then	#istruzione branch on greater_than_zero (se \$t0>0 , salto all'etichetta then)
	blt \$t0, \$zero, else	#istruzione branch on less_than_zero (se \$t0<0 , salto all'etichetta else)
	and \$s0, \$t1, \$t2	#eseguita solo se non sono verificate le due condizioni \$t0<0 e \$t0>0,
		#quindi nel caso dell'opzione \$t0=0 (metto il risultato in \$s0)
	j fine	#eseguita solo quando \$t0=0
then:		#scrive in \$s0 la somma dei 3 numeri con due operazioni add
	add \$s0, \$t0, \$t1	
	add \$s0, \$s0, \$t2	
	j fine	
else:		#scrive in \$s0 il prodotto dei 3 numeri con due operazioni mul
	mul \$s0, \$t0, \$t1	
	mul \$s0, \$s0, \$t2	
	j fine	
fine:	sw \$s0, 12(\$t7)	#scrittura in memoria del contenuto di \$s0 3 word (=3*4 byte) dopo il valore dell'etichetta "valori" che era stata caricata nel registro \$t7

Esercizio 6

Dato il seguente frammento di codice:

```
        li $t2, 8
ciclo:
        .....
        sw $t2, 0($t0)
        addi $t0, $t0, 4
        addi $t2, $t2, -1
        j ciclo
fine_ciclo:
        jr $ra
```

Quale delle seguenti istruzioni va collocata al posto dei puntini per scrivere i valori 8,7,6,5,4,3,2,1,0 nell'array di word indirizzato da \$t0?

1. bgtz \$t2, fine_ciclo
2. bltz \$t2, fine_ciclo
3. bgez \$t2, fine_ciclo
4. blez \$t2, fine_ciclo
5. beqz \$t2, fine_ciclo
6. bnez \$t2, fine_ciclo
7. nessuna delle altre risposte

Esercizio 6 – soluzione

Dato il seguente frammento di codice:

```
        li $t2, 8
ciclo:  .....
        sw $t2, 0($t0)
        addi $t0, $t0, 4
        addi $t2, $t2, -1
        j ciclo
fine_ciclo:
        jr $ra
```

Quale delle seguenti istruzioni va collocata al posto dei puntini per scrivere nell'array di word indirizzato da \$t0 i valori 8,7,6,5,4,3,2,1,0?

1. bgtz \$t2, fine_ciclo #branch se \$t2>0
2. **bltz \$t2, fine_ciclo** #branch se \$t2<0
3. bgez \$t2, fine_ciclo #branch se \$t2>0 o \$t2=0
4. blez \$t2, fine_ciclo #branch se \$t2<0 o \$t2=0
5. beqz \$t2, fine_ciclo #branch se \$t2=0
6. bnez \$t2, fine_ciclo #branch se \$t2<>0
7. nessuna delle altre risposte

Poichè vogliamo eseguire il ciclo 9 volte (e ad ogni ciclo decrementiamo \$t2 di 1 partendo da un valore iniziale di 8), il branch dovrà essere fatto la prima volta che \$t2 è minore di 0 mentre non dovrà essere fatto quando \$t2 è maggiore e anche quando è uguale a 0

Esercizio 7 – procedura foglia

Scrivere un programma assembly che calcola la somma di un array di numeri interi. Il programma deve prevedere una procedura che riceve come parametri la dimensione dell'array e l'indirizzo d'inizio dell'array, e ritornare la somma degli elementi dell'array

Esercizio 8 – procedure annidate

- Creare un file .asm con il data segment e il text segment contenuti nelle slide successive e provare ad eseguire step-by-step con QTSPIM
- In particolare osservare e verificare
 - i valori contenuti in PC e \$ra (prima e dopo l'esecuzione delle istruzioni jal e jr)
 - il valore contenuto in \$sp (ogni volta che viene usato)
 - il contenuto della porzione User stack della memoria dati (dopo ogni salvataggio di \$ra)

Esercizio 8 – data segment

#programma main e altre due procedure in cui vengono stampati messaggi che indicano il punto a cui si trova l'esecuzione del programma (all'inizio o subito dopo la fine di ogni procedura)

```
.data    # Dati del programma
        msg1: .asciiz "Esecuzione Main...\n"
        msg2: .asciiz "Esecuzione Procedura1...\n"
        msg3: .asciiz "Esecuzione Procedura2...\n"
        msg4: .asciiz "Fine Procedura2.\n"
        msg5: .asciiz "Fine Procedura1.\n"
        msg6: .asciiz "Fine Main.\n"
```

Esercizio 8 – text segment (main-originale)

.text

```
main:                #Codice di main. Anche main e' una procedura; al termine dovra' fare jr $ra al chiamante (il SO)
    li $v0, 4         # Stampa msg1 con system call (v. pag. A-44)
    la $a0, msg1
    syscall
    addi $sp, $sp, -4  #faccio spazio per salvare $ra, spostando $sp verso il basso
    sw $ra, 0($sp)     #salvo $ra prima di jal nello spazio allocato
    jal procedura1    # Jump And Link alla procedura procedura1; oltre a aggiornare PC con l'indirizzo di inizio di
                        #procedura1, viene salvato in $ra l'indirizzo dell'istruzione successiva (etichetta RIENTRODAPROCEDURA1)
RIENTRODAPROCEDURA1: li $v0, 4         # Stampa msg6
                    la $a0, msg6
                    syscall
                    lw $ra, 0($sp)      #ripristino $ra prima di jr $ra
                    addi $sp, $sp, 4    #riposiziono lo stack pointer liberando spazio di memoria
                    jr $ra             # salta a istruzione il cui indirizzo e' in registro $ra (perchè anche main è una
                                        #procedura)
```

Esercizio 8 – text segment (procedura1)

```
procedura1:                # Codice della procedura procedura1
    addi $sp, $sp, -4      # Faccio spazio nello stack (spostando $sp verso il basso)
    sw $ra, 0($sp)         # e ci salvo $ra, cioe' salvo il punto di rientro chiamante
    li $v0, 4              # Stampa msg2
    la $a0, msg2
    syscall
    jal procedura2        # Salta a (in PC viene messo l'indirizzo d'inizio di) procedura2 e in $ra viene messo l'indirizzo
                           # dell'istruzione successiva (corrispondente all'etichetta RIENTRODAPROCEDURA2)

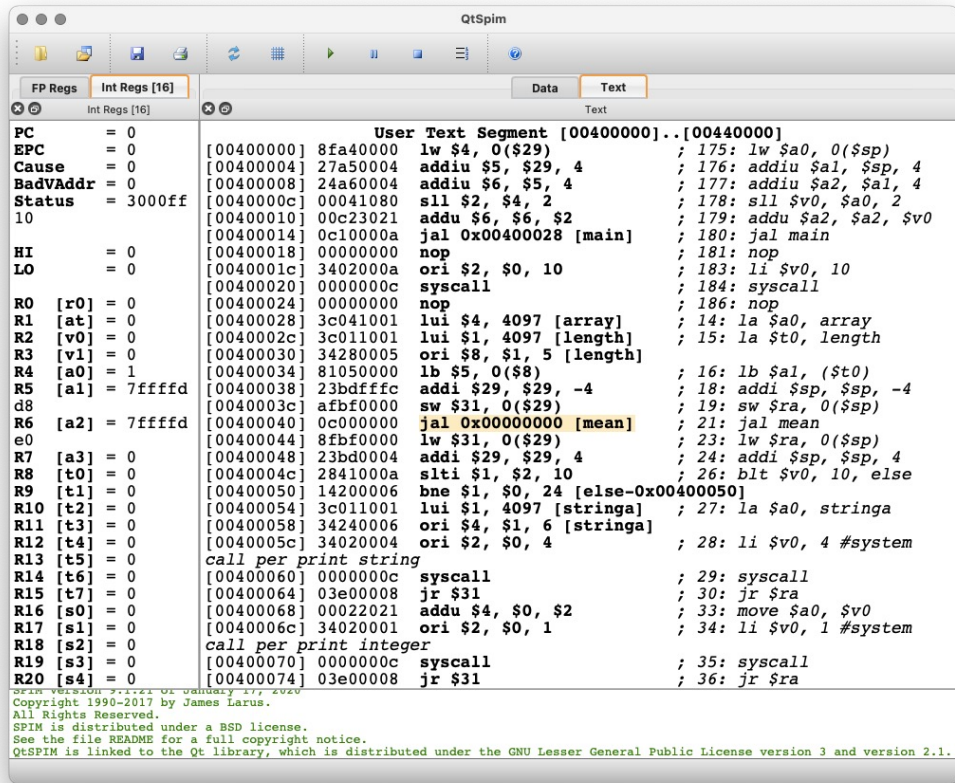
RIENTRODAPROCEDURA2:      li $v0, 4                # Stampa msg5
                           la $a0, msg5
                           syscall
                           lw $ra, 0($sp)          # Recupera il valore di $ra dallo stack corrispondente all'istruzione subito
                           #dopo la jal che ha chiamato la procedura, cioe' RIENTRODAPROCEDURA1
                           addi $sp, $sp, 4        # libera spazio allocato nello stack
                           jr $ra                 # salta a istruzione cui indirizzo e' in $ra
```

Esercizio 8 – text segment (procedura2)

```
procedura2:          # Codice della procedura procedura2 (foglia)
    li $v0, 4         # Stampa msg3
    la $a0, msg3
    syscall
    li $v0, 4         # Stampa msg4
    la $a0, msg4
    syscall
    jr $ra            # Restituisce il controllo al chiamante (ovvero fa
                    #jump all'indirizzo indicato da $ra cioè quello
                    #dell'istruzione successiva alla jal che ha chiamato la
                    #procedura corrispondente all'etichetta
                    #RIENTRODAPROCEDURA2)
```

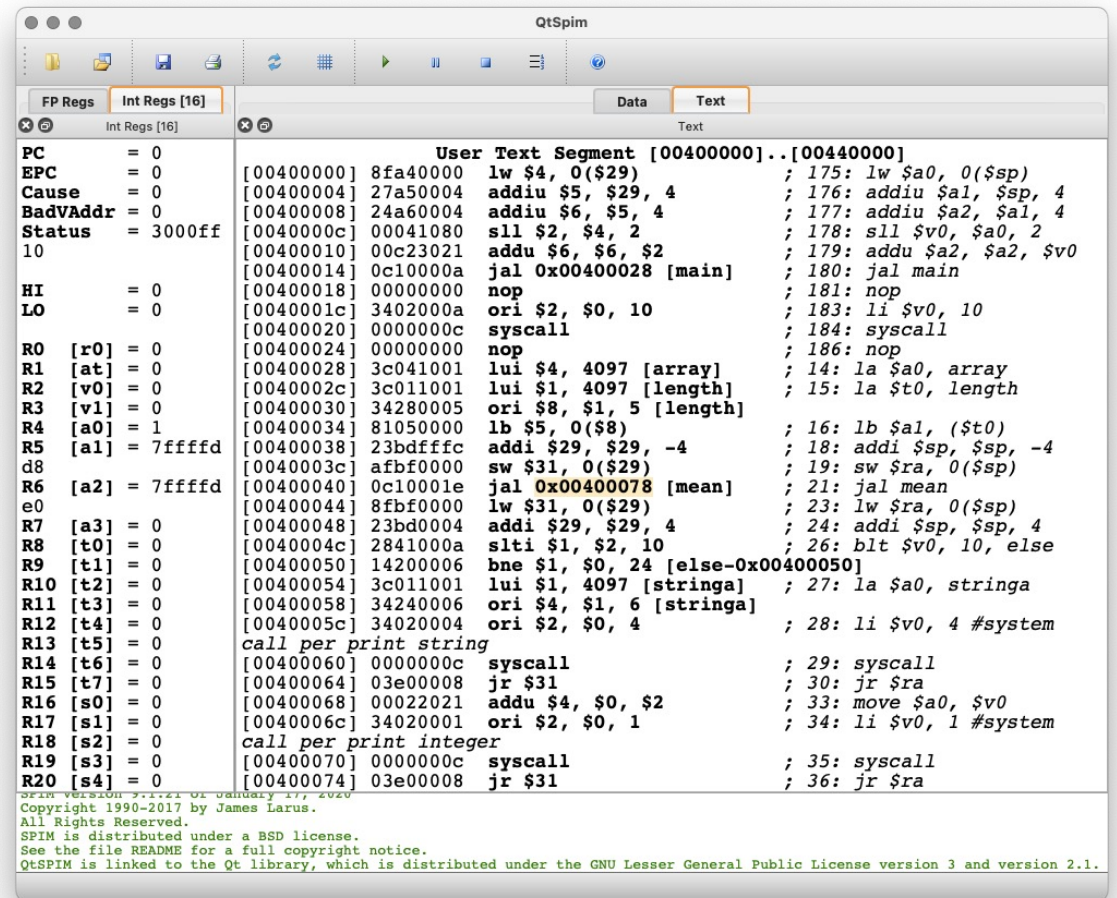
Altro esempio procedure annidate:

- vector_mean.asm...che usa mean.asm che usa sum.asm
- N.B. Valore etichette in jal dopo assemblaggio (e linking) vari moduli



```
PC = 0
EPC = 0
Cause = 0
BadVAddr = 0
Status = 3000ff
10
HI = 0
LO = 0
R0 [r0] = 0
R1 [at] = 0
R2 [v0] = 0
R3 [v1] = 0
R4 [a0] = 1
R5 [a1] = 7ffffd
d8
R6 [a2] = 7ffffd
e0
R7 [a3] = 0
R8 [t0] = 0
R9 [t1] = 0
R10 [t2] = 0
R11 [t3] = 0
R12 [t4] = 0
R13 [t5] = 0
R14 [t6] = 0
R15 [t7] = 0
R16 [s0] = 0
R17 [s1] = 0
R18 [s2] = 0
R19 [s3] = 0
R20 [s4] = 0

User Text Segment [00400000]..[00440000]
[00400000] 8fa40000 lw $4, 0($29) ; 175: lw $a0, 0($sp)
[00400004] 27a50004 addiu $5, $29, 4 ; 176: addiu $a1, $sp, 4
[00400008] 24a60004 addiu $6, $5, 4 ; 177: addiu $a2, $a1, 4
[0040000c] 00041080 sll $2, $4, 2 ; 178: sll $v0, $a0, 2
[00400010] 00c23021 addu $6, $6, $2 ; 179: addu $a2, $a2, $v0
[00400014] 0c10000a jal 0x00400028 [main] ; 180: jal main
[00400018] 00000000 nop ; 181: nop
[0040001c] 3402000a ori $2, $0, 10 ; 183: li $v0, 10
[00400020] 0000000c syscall ; 184: syscall
[00400024] 00000000 nop ; 186: nop
[00400028] 3c041001 lui $4, 4097 [array] ; 14: la $a0, array
[0040002c] 3c011001 lui $1, 4097 [length] ; 15: la $t0, length
[00400030] 34280005 ori $8, $1, 5 [length]
[00400034] 81050000 lb $5, 0($8) ; 16: lb $a1, ($t0)
[00400038] 23bdfffc addi $29, $29, -4 ; 18: addi $sp, $sp, -4
[0040003c] afbf0000 sw $31, 0($29) ; 19: sw $ra, 0($sp)
[00400040] 0c000000 jal 0x00000000 [mean] ; 21: jal mean
[00400044] 8fbf0000 lw $31, 0($29) ; 23: lw $ra, 0($sp)
[00400048] 23bd0004 addi $29, $29, 4 ; 24: addi $sp, $sp, 4
[0040004c] 2841000a slli $1, $2, 10 ; 26: blt $v0, 10, else
[00400050] 14200006 bne $1, $0, 24 [else-0x00400050]
[00400054] 3c011001 lui $1, 4097 [stringa] ; 27: la $a0, stringa
[00400058] 34240006 ori $4, $1, 6 [stringa]
[0040005c] 34020004 ori $2, $0, 4 ; 28: li $v0, 4 #system
[00400060] 0000000c syscall ; 29: syscall
[00400064] 03e00008 jr $31 ; 30: jr $ra
[00400068] 00022021 addu $4, $0, $2 ; 33: move $a0, $v0
[0040006c] 34020001 ori $2, $0, 1 ; 34: li $v0, 1 #system
[00400070] 0000000c syscall ; 35: syscall
[00400074] 03e00008 jr $31 ; 36: jr $ra
```



```
PC = 0
EPC = 0
Cause = 0
BadVAddr = 0
Status = 3000ff
10
HI = 0
LO = 0
R0 [r0] = 0
R1 [at] = 0
R2 [v0] = 0
R3 [v1] = 0
R4 [a0] = 1
R5 [a1] = 7ffffd
d8
R6 [a2] = 7ffffd
e0
R7 [a3] = 0
R8 [t0] = 0
R9 [t1] = 0
R10 [t2] = 0
R11 [t3] = 0
R12 [t4] = 0
R13 [t5] = 0
R14 [t6] = 0
R15 [t7] = 0
R16 [s0] = 0
R17 [s1] = 0
R18 [s2] = 0
R19 [s3] = 0
R20 [s4] = 0

User Text Segment [00400000]..[00440000]
[00400000] 8fa40000 lw $4, 0($29) ; 175: lw $a0, 0($sp)
[00400004] 27a50004 addiu $5, $29, 4 ; 176: addiu $a1, $sp, 4
[00400008] 24a60004 addiu $6, $5, 4 ; 177: addiu $a2, $a1, 4
[0040000c] 00041080 sll $2, $4, 2 ; 178: sll $v0, $a0, 2
[00400010] 00c23021 addu $6, $6, $2 ; 179: addu $a2, $a2, $v0
[00400014] 0c10000a jal 0x00400028 [main] ; 180: jal main
[00400018] 00000000 nop ; 181: nop
[0040001c] 3402000a ori $2, $0, 10 ; 183: li $v0, 10
[00400020] 0000000c syscall ; 184: syscall
[00400024] 00000000 nop ; 186: nop
[00400028] 3c041001 lui $4, 4097 [array] ; 14: la $a0, array
[0040002c] 3c011001 lui $1, 4097 [length] ; 15: la $t0, length
[00400030] 34280005 ori $8, $1, 5 [length]
[00400034] 81050000 lb $5, 0($8) ; 16: lb $a1, ($t0)
[00400038] 23bdfffc addi $29, $29, -4 ; 18: addi $sp, $sp, -4
[0040003c] afbf0000 sw $31, 0($29) ; 19: sw $ra, 0($sp)
[00400040] 0c10001e jal 0x00400078 [mean] ; 21: jal mean
[00400044] 8fbf0000 lw $31, 0($29) ; 23: lw $ra, 0($sp)
[00400048] 23bd0004 addi $29, $29, 4 ; 24: addi $sp, $sp, 4
[0040004c] 2841000a slli $1, $2, 10 ; 26: blt $v0, 10, else
[00400050] 14200006 bne $1, $0, 24 [else-0x00400050]
[00400054] 3c011001 lui $1, 4097 [stringa] ; 27: la $a0, stringa
[00400058] 34240006 ori $4, $1, 6 [stringa]
[0040005c] 34020004 ori $2, $0, 4 ; 28: li $v0, 4 #system
[00400060] 0000000c syscall ; 29: syscall
[00400064] 03e00008 jr $31 ; 30: jr $ra
[00400068] 00022021 addu $4, $0, $2 ; 33: move $a0, $v0
[0040006c] 34020001 ori $2, $0, 1 ; 34: li $v0, 1 #system
[00400070] 0000000c syscall ; 35: syscall
[00400074] 03e00008 jr $31 ; 36: jr $ra
```